

Problem Landscape and Success Criteria for One-User Offline-to-Online Personalized Policies

Research Position and Evaluation Contract

August 26, 2026

1 Setting

Evidence base. Two project PRISMA runs yielded 6,663 raw records and 119 full-text reviews. We cite primary papers for their methods and evaluation settings, not as evidence that our policy works.

One user is the unit of personalization. For user u , the offline history

$$H_u^{(0)} = \{\text{thoughts, conversations, corrections, preferences, task states, actions, tools, and outcomes}\}$$

produces an initial checkpoint $c_u^{(0)}$. After deployment, interaction batch $D_u^{(t)}$ is generated by the active checkpoint $c_u^{(t)}$. The update is

$$c_u^{(t+1)} = \mathcal{A}(c_u^{(t)}, H_u^{(0)}, D_u^{(1:t)}). \quad (1)$$

Each record stores its user, time, task, checkpoint, trace, feedback source, and training eligibility. A shared prior is allowed; raw histories remain separate.

A policy update is required. Retrieval, profiles, memory, and decoding control are baselines, not the learned object. OPPU trains a private user adapter [Tan et al., 2024]; P-RLHF trains a user-conditioned policy [Li et al., 2024]; and SPRInG updates a user adapter over time [Kim and Kim, 2026]. CoSteer is tuning-free [Lv et al., 2025], while FABLE learns an external execution policy around a frozen agent [Jin et al., 2026]. Neither CoSteer nor FABLE updates a deployed user’s language-model policy weights.

Objective. Improve personalization without degrading task success or safety:

$$\begin{aligned} \max_{\mathcal{A}} \quad & \mathbb{E}[U_{\text{pers}}(c_u^{(t+1)})] \\ \text{s.t.} \quad & \mathbb{E}[U_{\text{task}}(c_u^{(t+1)}) - U_{\text{task}}(c_u^{(t)})] \geq -\delta_{\text{task}}, \\ & \mathbb{E}[U_{\text{safe}}(c_u^{(t+1)}) - U_{\text{safe}}(c_u^{(t)})] \geq -\delta_{\text{safe}}, \\ & C_{\text{train}}, C_{\text{serve}}, C_{\text{feedback}} \text{ remain within declared budgets.} \end{aligned} \quad (2)$$

Exact scope. The target is one user-specific model that remains in use over time. It is first trained on that user’s history, then updated from later interactions with the same user. It executes agent tasks, and learning changes its language-model parameters. A *live update* happens after the system is already in use: new interactions change the model, and the new model replaces the old one. Replaying a fixed dataset or collecting data during one training run is not a live update. Here, a personalized agent executes tasks for a user by choosing tools, using memory, taking actions, or asking questions. Personalized text generation alone is not agent execution. No cited method covers all of these properties.

2 How Prior Work Differs

2.1 Learning the user

P1: identify preferences from sparse, mixed evidence. A user supplies requests, choices, edits, and task outcomes, not one uniform label. Population training can blur minority preferences [Li et al., 2024, Jang et al., 2023]. CoPL and PROPER share structure across users [Choi et al., 2025, Zhang et al., 2025]; FSPO adapts from few examples [Singh et al., 2025]. We instead use one user’s full history.

Comparison. These methods learn from stored preference data. Several combine data across users, and CoPL trains a reward model rather than the LLM that produces responses. They do not study an agent that learns from one user’s later task interactions.

P2: separate lasting preferences from temporary requests. A request may express a lasting preference, a task constraint, or noise. SPRInG detects likely preference changes before updating a user adapter [Kim and Kim, 2026]. In an agent, a bad result may come from a tool or reasoning error rather than a wrong preference.

Comparison. SPRInG trains a separate LoRA for each user and updates its weights over a fixed sequence of later text records. It personalizes text generation, not a system that executes tasks through tools, memory, actions, or questions. It therefore does not test live learning from agent actions and task results.

P3: learn from little user history without mixing users. Collaborative and federated models improve sparse-user adaptation [Choi et al., 2025, Zhang et al., 2025, Zhu et al., 2026]. Our setting may use a shared base model, but histories, updates, and evaluation remain user-specific.

Comparison. These methods learn user-specific components from stored data for many users. PROPER and FedPDPO update LLM components, while CoPL updates a reward model. They do not learn from a deployed agent. Our setting starts with a known user, so learning with no history is not required.

2.2 Moving from offline history to live learning

P4: build an offline model that can still improve later. The offline model must complete tasks and still be able to learn. OPPU provides a starting model for each user [Tan et al., 2024]. WebRL shows that an agent learns better when it starts from a working model [Qi et al., 2025].

Comparison. OPPU trains an LLM adapter for one user but stops after offline training. WebRL trains an agent and then improves it with new data collected during training, but it has no individual user. Neither tests a deployed user-specific agent that keeps learning.

P5: move from offline to live training without losing performance. Offline records come from older models. Bridging studies when to copy new weights to the model that collects data [Lanchantin et al., 2025]. Hybrid preference learning combines stored and newly requested comparisons [Bose et al., 2024]. Policy expansion keeps the old policy available while the new one explores [Zhang et al., 2023]. We must also record the user, feedback source, and model version for every interaction.

Comparison. These methods start with offline data and later collect new data during training. Bridging updates LLM weights; the other papers study more general optimization methods. None studies one user’s deployed agent.

P6: learn changing preferences without forgetting stable ones. SPRInG selects likely preference changes and keeps difficult past examples [Kim and Kim, 2026]. Stable Asynchrony learns from records produced by older model versions [Huang et al., 2026]. We need both while retaining stable preferences.

Comparison. SPRInG models one user but uses stored text. Stable Asynchrony updates an LLM from records produced by older model versions, but has no user personalization or deployed agent. Neither studies preference changes in a deployed personal agent.

P7: learn from fewer user corrections. User feedback is expensive. Hybrid preference optimization trades offline data against new queries [Bose et al., 2024]. FABLE learns a per-user execution policy only during live use [Jin et al., 2026]. We measure interactions needed to reach the same score on future tasks.

Comparison. FABLE learns online for one user while the host agent stays unchanged and executes the task. It has no learned offline starting point and does not update the host LLM. Hybrid preference optimization combines stored data with new queries, but is not a personalized agent. Neither trains one personalized LLM across both phases.

2.3 Learning an agent policy

P8: personalize agent behavior without reducing task success. Personalized agents choose tools, memory, clarifications, and response style while completing the task. PersonalWAB studies web actions [Cai et al., 2025], FingerTip 20K studies mobile execution [Yang et al., 2026], and FABLE adapts execution around an agent that stays unchanged [Jin et al., 2026].

Comparison. PersonalWAB and FingerTip evaluate personalized actions on fixed benchmarks. FABLE learns an external controller online for each user. None first trains a personal LLM from past user data and then continues updating that LLM after deployment.

P9: identify which action caused success or failure. A correction may follow many reasoning and tool steps. WebRL learns from a success or failure signal given only at the end [Qi et al., 2025]; we must also identify which steps reveal the user’s preference.

Comparison. WebRL updates an LLM web agent from task success and failure, but it has no individual user and collects data during training. Our method must also learn which action reflects the user’s preference.

2.4 Obtaining credible and affordable evidence

P10: avoid learning quirks of simulated users. UserRL, UserVille, ProPerSim, and CollabLLM train or evaluate agents with simulated users [Qian et al., 2025, Sun et al., 2025, Kim et al., 2025, Wu et al., 2025]. PersonalLLM creates synthetic preference models [Zollo et al., 2024]. Simulators reduce cost but can impose their own style and reward biases [Rezk et al., 2025]; real-user evidence remains necessary.

Comparison. Several papers train or evaluate agents with simulated personas, but they do not maintain a policy for one real user and update it during live use. Simulation can support training; it cannot prove improvement for a deployed user.

P11: reduce the cost of one model per user. Per-user adapters and live training add storage and compute. OPPU uses lightweight adapters [Tan et al., 2024], CoSteer uses a local model to guide a cloud model [Lv et al., 2025], and FABLE learns a small external policy [Jin et al., 2026]. We test whether a compact user policy can match a larger generic model at lower serving cost.

Comparison. OPPU trains a user’s LLM adapter offline. CoSteer changes generation without training model weights. FABLE learns an external controller during live use. None trains a compact user LLM offline and then continues updating it during live use.

P12: test future behavior, not memory of past text. LongLaMP provides chronological long-form tasks [Kumar et al., 2024]; PrefEval tests preference following [Zhao et al., 2025]; PERMA adds evolving preferences and tasks [Liu et al., 2026]; and PRISM supplies individual feedback [Kirk et al., 2024]. Our endpoint is behavior on untouched future tasks.

Comparison. These are datasets and benchmarks, not training methods. They contain per-user histories; PERMA also evaluates task execution. None updates a deployed model.

3 Claim

A compact, user-owned language model first learns from one person’s past conversations, corrections, and tasks. It then continues learning from that person’s later agent interactions by updating its weights, without reducing task success or safety.

This requires an offline personal model, later parameter updates, and a comparison on the same future tasks. The method specifies how past user data, new task records, feedback, replay, and model replacement work together.

3.1 Additional claims

Report these claims only when their tests pass:

Fast adaptation. Fewer real interactions reach the same valid score.

Value of task traces. Tool actions and verified outcomes improve future execution beyond text-only history.

Small versus large. A compact personalized policy beats a larger generic model while preserving task success and lowering cost.

Simulation benefit. Simulator training reduces real-user interactions and survives held-out human evaluation.

Privacy. The implementation follows its declared data boundary without exposing another user’s raw records.

4 Success Criteria

4.1 Required checks

Before comparing scores, every reported update must pass:

1. the parent and candidate checkpoint hashes differ;
2. the candidate loads in a separate worker and produces the evaluated task records;
3. offline training, live updates, validation, and future testing use separate time periods;
4. every training record stores the user, task, generating model, timestamp, and feedback source;
5. failed runs, users, and rollbacks remain in the report;
6. the simulator, reward model, or LLM judge has not seen the future test or its answer.

If any check fails, the run is invalid.

4.2 Main statistical test

Let b be the strongest baseline selected on validation data. Compare it with the candidate on the same future tasks:

$$\Delta_{\text{pers}} = U_{\text{pers}}(c_{\text{candidate}}) - U_{\text{pers}}(c_b), \quad (3)$$

$$\Delta_{\text{task}} = U_{\text{task}}(c_{\text{candidate}}) - U_{\text{task}}(c_b). \quad (4)$$

Table 1: Success criteria. The first four rows test the main claim.

Claim	Measurement	Pass condition
Personalized improvement	Paired future-task Δ_{pers} against the strongest baseline.	Lower 95% bound is above zero and the gain exceeds the value chosen before evaluation.
Task preservation	Paired task success or environment reward.	Lower 95% bound exceeds $-\delta_{\text{task}}$.
Value of live updates	Updated model versus the unchanged offline model and a simple cumulative update.	Candidate passes equation (5) against both.
Real policy change	Parameter count, hashes, load receipt, and changed outputs.	Nonzero finite update; candidate is loaded for evaluation.
Fast adaptation	Learning curve and interactions needed to reach the target.	Better curve or fewer interactions to the same target.
Retention	Performance on earlier stable preferences and tasks.	Task success stays within its margin and forgetting is lower than simple continual training.
Preference-change recovery	Score after a lasting preference change.	Faster recovery without extra updates for temporary noise.
Safety	Safety failures on a fixed test set.	No increase in severe events; other changes stay within the chosen margin.
Small versus large	Compact personalized model versus large generic, RAG, and profile-enabled models under equal information.	Better personalization, preserved task success, and lower serving cost.
Simulator benefit	Agreement with people and ranking of compared methods.	The gain survives held-out human evaluation.
Efficiency	Training GPU-hours, inference latency, memory, storage, tokens, API cost, and user-feedback count.	No compared method is both better and cheaper under matched budgets.
Ownership/privacy	Where raw records are sent, communication volume, and cross-user leakage.	No raw history crosses the declared boundary.

The main claim passes only when

$$\text{LCB}_{95\%}(\Delta_{\text{pers}}) > 0 \quad \text{and} \quad \text{LCB}_{95\%}(\Delta_{\text{task}}) \geq -\delta_{\text{task}}. \quad (5)$$

Choose the minimum useful personalization gain and δ_{task} before final evaluation. Group confidence intervals by user or related task; repeated turns are not independent samples.

4.3 Large-model comparison

The large-model comparison must include:

1. a large generic model without personal history;
2. the same model with the full allowed history;
3. strong RAG or profile prompting;
4. a method that changes generation without training, such as CoSteer, when executable [Lv et al., 2025];
5. the compact offline-only model and the same model after live updates.

Match prompts, tools, history, and task budgets. A compact model wins only if it improves personalization, preserves task success, and costs less.

Table 2: Minimum baseline families.

Family	Purpose	Representative implementation
No personalization	Measures the value of any user information.	Same-size base and large API model.
Prompt/profile/RAG	Tests whether parameter updates are necessary.	Full-history prompting, retrieval, and generated profile.
Offline per-user PEFT	Tests the value of live updates.	OPPU [Tan et al., 2024].
Shared personalized policy	Tests one-user ownership against a population model.	P-RLHF, CoPL, or PROPER [Li et al., 2024, Choi et al., 2025, Zhang et al., 2025].
Repeated update for each user	Tests the proposed update and replay rule.	SPRInG and simple cumulative SFT [Kim and Kim, 2026].
Live updates only	Measures the value of offline initialization.	Same optimizer initialized from the base model.
Simple offline-to-live update	Tests whether the proposed update rule is necessary.	Sequential SFT/DPO/RL with no special replay or model-switching rule.
Live control without LLM updates	Tests whether policy training is worth its cost.	FABLE and CoSteer [Jin et al., 2026, Lv et al., 2025].
Large-model teacher	Tests compact-model competitiveness.	The CLI/API model that produced the offline task records.

4.4 Adaptation speed

For threshold τ , measure real interactions rather than training passes:

$$T_\tau = \min\{n : U_{\text{pers}}(c_u^{(n)}) \geq \tau \text{ and task/safety gates pass}\}. \quad (6)$$

Also report scores throughout learning, the final score, clarifications, correction time, and rollbacks.

4.5 Simulator evidence

Test:

1. **Preference match:** agreement on held-out real-user choices.
2. **Method ranking:** whether simulated and real users rank methods in the same order.
3. **Training value:** whether simulation reduces later real-user interactions.

Simulator reward alone does not support a real-user claim.

5 Baselines and Evaluation

Evaluation benchmarks. Use a chronological stream such as LongLaMP [Kumar et al., 2024] and an executable agent benchmark such as PersonalWAB or FingerTip 20K [Cai et al., 2025, Yang et al., 2026]. PrefEval and PERMA add long-horizon tests [Zhao et al., 2025, Liu et al., 2026]. Final results need held-out human feedback or stored real-user records and two different benchmarks.

Replication. Repeat the protocol across independent users. Report per-user results, the median, worst decile, and confidence intervals grouped by user, with at least three runs for early tests and five for final results.

6 Claims We Must Not Make

- Replaying old records in time order is not live learning after deployment.

- Better retrieval or profile recall is not evidence of a better policy.
- A simulator-only win is not evidence of real-user improvement.
- A changed loss value does not show that the new model was loaded.
- A personalized reward-model gain is not automatically a policy or task gain [Rezk et al., 2025].
- Keeping adapters local does not by itself prove privacy.
- Beating a large model on style does not mean beating it on task capability.
- Mean improvement cannot hide severe failures for a subset of users.

References

- Avinandan Bose, Zhihan Xiong, Aadirupa Saha, Simon Shaolei Du, and Maryam Fazel. Hybrid preference optimization for alignment: Provably faster convergence rates by combining offline preferences with on-line exploration. *arXiv preprint arXiv:2412.10616*, 2024. URL <https://arxiv.org/abs/2412.10616>.
- Hongru Cai, Yongqi Li, Wenjie Wang, Fengbin Zhu, Xiaoyu Shen, Wenjie Li, and Tat-Seng Chua. Large language models empowered personalized web agents. In *Proceedings of the ACM Web Conference*, 2025. doi: 10.1145/3696410.3714842. URL <https://arxiv.org/abs/2410.17236>.
- Youngbin Choi, Seunghyuk Cho, Minjong Lee, MoonJeong Park, Yesong Ko, Jungseul Ok, and Dongwoo Kim. CoPL: Collaborative preference learning for personalizing LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. doi: 10.18653/v1/2025.emnlp-main.650. URL <https://aclanthology.org/2025.emnlp-main.650/>.
- Luke J. Huang, Zhuoyang Zhang, Qinghao Hu, Shangdong Yang, and Song Han. Stable asynchrony: Variance-controlled off-policy RL for LLMs. *arXiv preprint arXiv:2602.17616*, 2026. URL <https://arxiv.org/abs/2602.17616>.
- Joel Jang, Seungone Kim, Bill Yuchen Lin, Yizhong Wang, Jack Hessel, Luke Zettlemoyer, Hannaneh Hajishirzi, Yejin Choi, and Prithviraj Ammanabrolu. Personalized soups: Personalized large language model alignment via post-hoc parameter merging. *arXiv preprint arXiv:2310.11564*, 2023. URL <https://arxiv.org/abs/2310.11564>.
- Dian Jin, Zhi Zhang, Huichao Li, Yihe Pan, Rundong Huang, and Doudou Zhou. Personalizing large language model agents with small policy models. *arXiv preprint arXiv:2608.00215*, 2026. URL <https://arxiv.org/abs/2608.00215>.
- Jiho Kim, Junseong Choi, Woosog Chay, Daeun Kyung, Yeonsu Kwon, Yohan Jo, and Edward Choi. ProPerSim: Developing proactive and personalized AI assistants through user-assistant simulation. *arXiv preprint arXiv:2509.21730*, 2025. URL <https://arxiv.org/abs/2509.21730>.
- Seoyeon Kim and Jaehyung Kim. SPRInG: Continual LLM personalization via selective parametric adaptation and retrieval-interpolated generation. *arXiv preprint arXiv:2601.09974*, 2026. URL <https://arxiv.org/abs/2601.09974>.
- Hannah Rose Kirk, Alexander Whitefield, Paul Roettger, Andrew Bean, Katerina Margatina, Juan Ciro, Rafael Mosquera, Max Bartolo, Adina Williams, He He, Bertie Vidgen, and Scott A. Hale. The PRISM alignment dataset: What participatory, representative and individualised human feedback reveals about the subjective and multicultural alignment of large language models. *arXiv preprint arXiv:2404.16019*, 2024. URL <https://arxiv.org/abs/2404.16019>.
- Ishita Kumar, Snigdha Viswanathan, Sushrita Yerra, Alireza Salemi, Ryan A. Rossi, Franck Dernoncourt, Hanieh Deilamsalehy, Xiang Chen, Ruiyi Zhang, Shubham Agarwal, Nedim Lipka, Chien Van Nguyen, Thien Huu Nguyen, and Hamed Zamani. LongLaMP: A benchmark for personalized long-form text generation. *arXiv preprint arXiv:2407.11016*, 2024. URL <https://arxiv.org/abs/2407.11016>.

- Jack Lanchantin, Angelica Chen, Janice Lan, Xian Li, Swarnadeep Saha, Tianlu Wang, Jing Xu, Ping Yu, Weizhe Yuan, Jason Weston, Sainbayar Sukhbaatar, and Ilia Kulikov. Bridging offline and online reinforcement learning for LLMs. *arXiv preprint arXiv:2506.21495*, 2025. URL <https://arxiv.org/abs/2506.21495>.
- Xinyu Li, Ruiyang Zhou, Zachary C. Lipton, and Leqi Liu. Personalized language modeling from personalized human feedback. *arXiv preprint arXiv:2402.05133*, 2024. URL <https://arxiv.org/abs/2402.05133>.
- Shuochen Liu, Junyi Zhu, Longhui Shu, Jun Lin, Yuhao Chen, Haotian Zhang, Chao Zhang, Derong Xu, Jia Li, Bo Tang, Zhiyu Li, Feiyu Xiong, Enhong Chen, and Tong Xu. PERMA: Benchmarking personalized memory agents via event-driven preference and realistic task environments. *arXiv preprint arXiv:2603.23231*, 2026. URL <https://arxiv.org/abs/2603.23231>.
- Hang Lv, Sheng Liang, Hao Wang, Hongchao Gu, Yaxiong Wu, Wei Guo, Defu Lian, Yong Liu, and Enhong Chen. CoSteer: Collaborative decoding-time personalization via local delta steering. *arXiv preprint arXiv:2507.04756*, 2025. URL <https://arxiv.org/abs/2507.04756>.
- Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiada Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2411.02337>.
- Cheng Qian, Zuxin Liu, Akshara Prabhakar, Jielin Qiu, Zhiwei Liu, Haolin Chen, Shirley Kokane, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. UserRL: Training interactive user-centric agent via reinforcement learning. *arXiv preprint arXiv:2509.19736*, 2025. URL <https://arxiv.org/abs/2509.19736>.
- Fady Rezk, Yuangang Pan, Chuan-Sheng Foo, Xun Xu, Nancy Chen, Henry Gouk, and Timothy Hospedales. The reward model selection crisis in personalized alignment. *arXiv preprint arXiv:2512.23067*, 2025. URL <https://arxiv.org/abs/2512.23067>.
- Anikait Singh, Sheryl Hsu, Kyle Hsu, Eric Mitchell, Stefano Ermon, Tatsunori Hashimoto, Archit Sharma, and Chelsea Finn. FSPO: Few-shot optimization of synthetic preferences personalizes to real users. *arXiv preprint arXiv:2502.19312*, 2025. URL <https://arxiv.org/abs/2502.19312>.
- Weiwei Sun, Xuhui Zhou, Weihua Du, Xingyao Wang, Sean Welleck, Graham Neubig, Maarten Sap, and Yiming Yang. Training proactive and personalized LLM agents. *arXiv preprint arXiv:2511.02208*, 2025. URL <https://arxiv.org/abs/2511.02208>.
- Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. doi: 10.18653/v1/2024.emnlp-main.372. URL <https://arxiv.org/abs/2402.04401>.
- Shirley Wu, Michel Galley, Baolin Peng, Hao Cheng, Gavin Li, Yao Dou, Weixin Cai, James Zou, Jure Leskovec, and Jianfeng Gao. CollabLLM: From passive responders to active collaborators. *arXiv preprint arXiv:2502.00640*, 2025. URL <https://arxiv.org/abs/2502.00640>.
- Qinglong Yang, Haoming Li, Haotian Zhao, Xiaokai Yan, Jingtao Ding, Fengli Xu, and Yong Li. FingerTip 20K: A benchmark for proactive and personalized mobile LLM agents. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2507.21071>.
- Haichao Zhang, Wei Xu, and Haonan Yu. Policy expansion for bridging offline-to-online reinforcement learning. *arXiv preprint arXiv:2302.00935*, 2023. URL <https://arxiv.org/abs/2302.00935>.

- Linhai Zhang, Jialong Wu, Deyu Zhou, and Yulan He. PROPER: A progressive learning framework for personalized large language models with group-level adaptation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. doi: 10.18653/v1/2025.acl-long.800. URL <https://arxiv.org/abs/2503.01303>.
- Siyan Zhao, Mingyi Hong, Yang Liu, Devamanyu Hazarika, and Kaixiang Lin. Do LLMs recognize your preferences? evaluating personalized preference following in LLMs. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2502.09597>.
- Kewen Zhu, Liping Yi, Zhiming Zhao, Zhuang Qi, Han Yu, and Qinghua Hu. FedPDPO: Federated personalized direct preference optimization for large language model alignment. *arXiv preprint arXiv:2603.19741*, 2026. URL <https://arxiv.org/abs/2603.19741>.
- Thomas P. Zollo, Andrew Wei Tung Siah, Naimeng Ye, Ang Li, and Hongseok Namkoong. PersonalLLM: Tailoring LLMs to individual preferences. *arXiv preprint arXiv:2409.20296*, 2024. URL <https://arxiv.org/abs/2409.20296>.